VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Dyuthi (1BM23CS097)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Dyuthi (1BM22CS097),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Chaitra D Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	4-2-2026	Write a python program to import and export data using Pandas library functions	
2	11-2-2026	Demonstrate various data pre-processing techniques for a given dataset	
3	25-2-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	
4	25-3-2026	Build Logistic Regression Model for a given dataset	
5	11-2-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	
6	25-2-2026	Build KNN Classification model for a given dataset.	
7	01-04-2026	Build Support vector machine model for a given dataset	
8	01-04-2026	Implement Random forest ensemble method on a given dataset.	
9	01-04-2026	Implement Boosting ensemble method on a given dataset.	
10	01-04-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	
11	08-04-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	

Github Link:
https://github.com/Dyuthiprasad/ML_Lab.git

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot

```
import pandas as pd
from sklearn.datasets import load_iris, load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
iris = load_iris()
x = iris.data
y = iris.target
x_train, x_test, y_train, y_test = train_test_split (x, y, test_size = 0.3,
                                                     random_state = 42)
model = DecisionTreeClassifier(criterion = 'entropy')
model.fit (x_train, y_train)
y_pred = model.predict (x_test)
print ("IRIS")
print ("Accuracy:", accuracy_score (y_test, y_pred))
print (classification_report (y_test, y_pred))

wine = load_wine()
x = wine.data
y = wine.target
x_train, x_test, y_train, y_test = train_test_split ( x, y, test_size = 0.3,
                                                      random_state = 42)
model = DecisionTreeClassifier(criterion = 'entropy')
model.fit (x_train, y_train)
y_pred = model.predict (x_test)
print ("WINE")
print ("Accuracy wine:" accuracy_score(y_test, y_pred))
print (classification_report (y_test, y_pred))
```

```python
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```python
df.shape
```

```
(150, 5)
```

```python
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```python
df.describe()
```

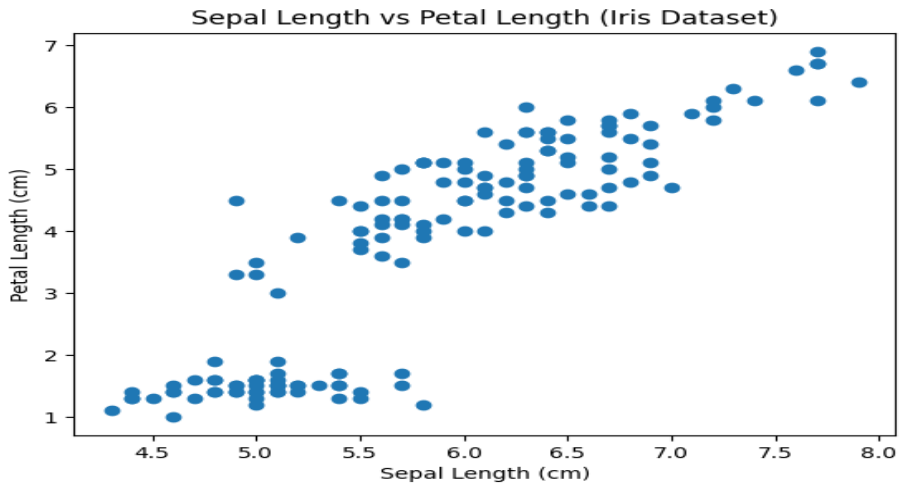
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```




```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```
df.corr()
```

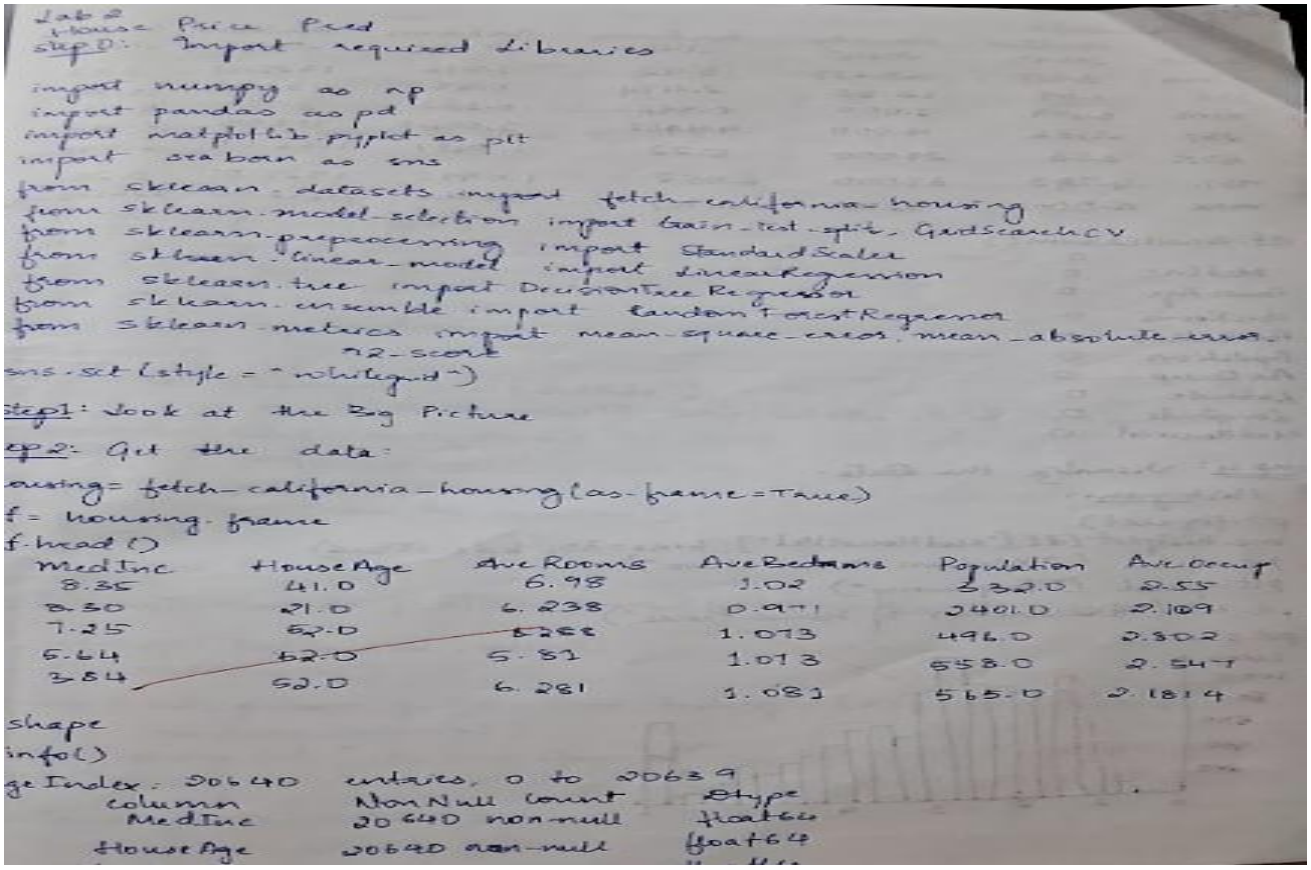
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

```
df.sort_values(by='petal length (cm)').head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 2

Demonstrate various data pre-processing techniques for a given dataset
Screenshot



Lab 2
House Price Pred
Step 0: Import required libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    r2_score
sns.set(style="whitegrid")
```

Step1: Look at the Big Picture

Step2: Get the data:

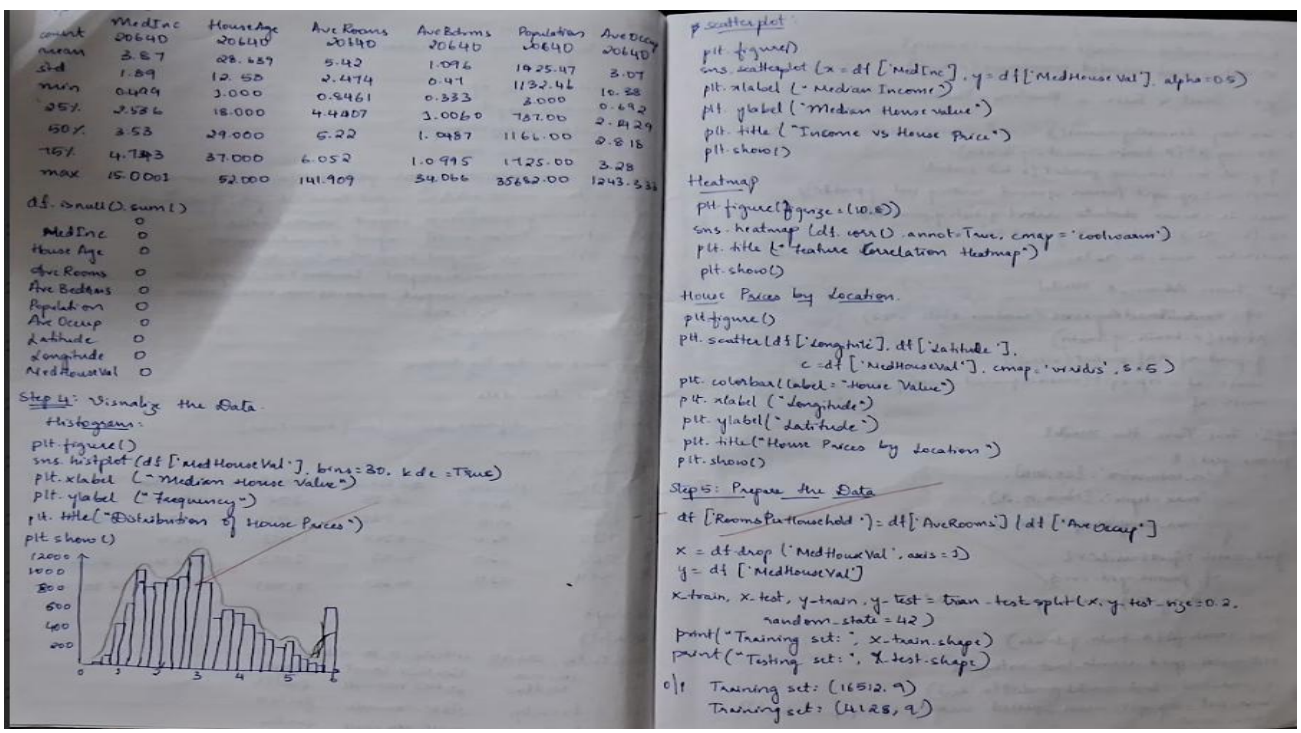
```
housing = fetch_california_housing(as_frame=True)
df = housing.frame
df.head()
```

MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup
8.35	41.0	6.98	1.02	322.0	2.55
8.30	21.0	6.238	0.971	2401.0	2.109
7.25	52.0	8.288	1.073	496.0	2.802
5.64	52.0	5.81	1.073	558.0	2.547
3.84	52.0	6.281	1.081	565.0	2.1814

```
df.shape
df.info()
```

ge Index: 20640 entries, 0 to 20639

Column	Non-Null Count	Dtype
MedInc	20640 non-null	float64
HouseAge	20640 non-null	float64



	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup
count	20640	20640	20640	20640	20640	20640
mean	3.87	28.639	5.42	1.096	1425.47	3.07
std	1.89	12.58	2.474	0.47	1132.46	10.38
min	0.499	1.000	0.8461	0.333	3.000	0.692
25%	2.536	18.000	4.4407	1.0060	787.00	2.8129
50%	3.53	29.000	5.22	1.0487	1166.00	2.818
75%	4.743	37.000	6.052	1.0995	1725.00	3.28
max	15.0001	52.000	141.909	34.066	35682.00	1243.333

```
df.isnull().sum()
```

	0
MedInc	0
HouseAge	0
AveRooms	0
AveBedrms	0
Population	0
AveOccup	0
Latitude	0
Longitude	0
MedHouseVal	0

Step 4: Visualize the Data

Histograms:

```
plt.figure()
sns.histplot(df['MedHouseVal'], bins=30, kde=True)
plt.xlabel("Median House Value")
plt.ylabel("Frequency")
plt.title("Distribution of House Prices")
plt.show()
```

```
# scatterplot
plt.figure()
sns.scatterplot(x=df['MedInc'], y=df['MedHouseVal'], alpha=0.5)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Income vs House Price")
plt.show()
```

Heatmap

```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()
```

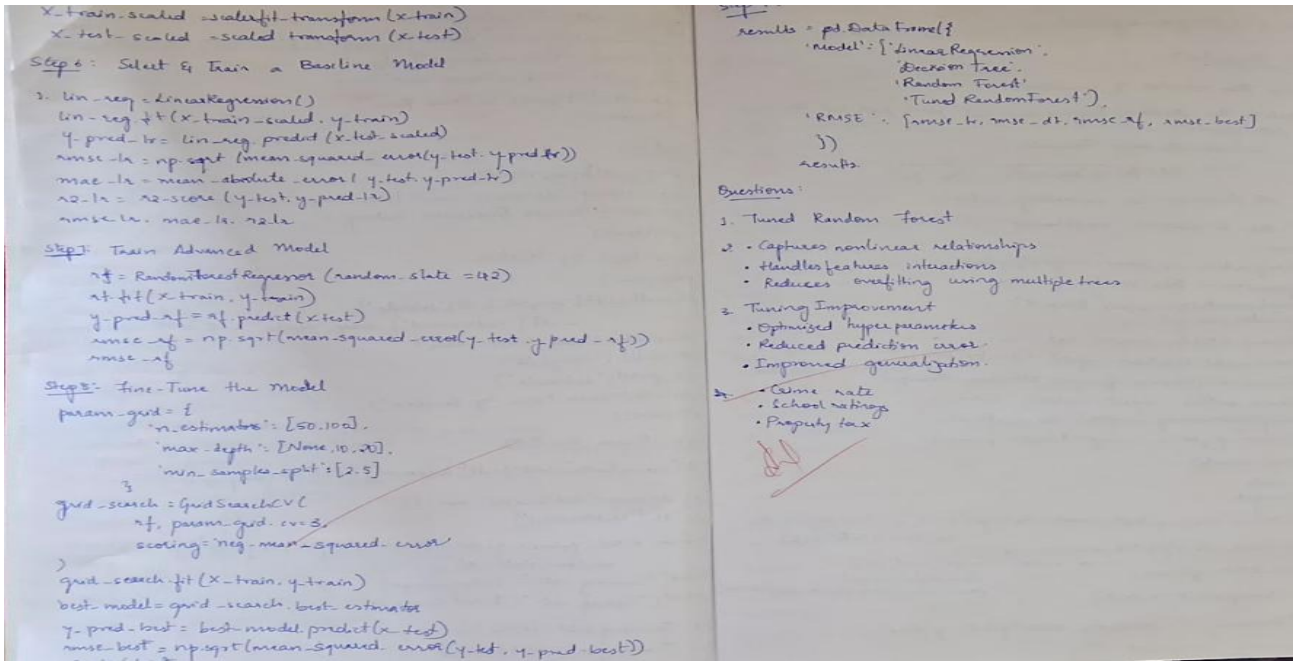
House Prices by Location.

```
plt.figure()
plt.scatter(df['Longitude'], df['Latitude'],
            c=df['MedHouseVal'], cmap='viridis', s=5)
plt.colorbar(label="House Value")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("House Prices by Location")
plt.show()
```

Step5: Prepare the Data

```
df['RoomsPerHousehold'] = df['AveRooms'] / df['AveOccup']
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

o/p Training set: (16512, 9)
Training set: (4128, 9)



Code:

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df.shape
```

```
(150, 5)
```

```
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```python
df.describe()
```

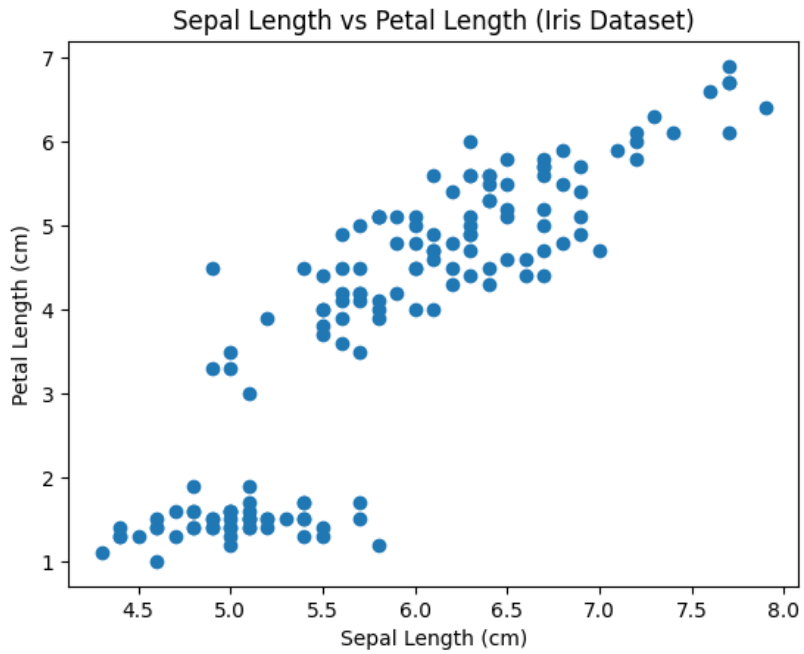
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```




```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```python
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

```python
df.sort_values(by='petal length (cm)').head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample

11/03/26 DECISION TREE CLASSIFICATION (ID3)

```
import pandas as pd
from sklearn.datasets import load_iris, load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
iris = load_iris()
x = iris.data
y = iris.target
x_train, x_test, y_train, y_test = train_test_split (x, y, test_size = 0.3,
                                                     random_state = 42)
model = DecisionTreeClassifier(criterion = 'entropy')
model.fit (x_train, y_train)
y_pred = model.predict (x_test)
print ("IRIS")
print ("Accuracy:", accuracy_score (y_test, y_pred))
print (classification_report (y_test, y_pred))

wine = load_wine()
x = wine.data
y = wine.target
x_train, x_test, y_train, y_test = train_test_split ( x, y, test_size = 0.3,
                                                      random_state = 42)
model = DecisionTreeClassifier(criterion = 'entropy')
model.fit (x_train, y_train)
y_pred = model.predict (x_test)
print ("WINE")
print ("Accuracy wine:" accuracy_score(y_test, y_pred))
print (classification_report (y_test, y_pred))
```

O/P

IRIS

Accuracy : 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

WINE:

Accuracy: 0.85185

	precision	recall	f1-score	support
0	0.82	0.95	0.88	19
1	0.86	0.86	0.86	21
2	0.91	0.71	0.80	14
accuracy			0.85	54
macro avg	0.86	0.84	0.85	54
weighted avg	0.86	0.85	0.85	54

Code:

```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(criterion='entropy', random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

new_sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_sample)

print("Predicted class:", prediction[0])
print("Predicted species:", iris.target_names[prediction][0])
```

```
Accuracy: 1.0
Predicted class: 0
Predicted species: setosa
```

Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset
Screenshot

KNN & Multilinear regression:

KNN (Algorithm)

1. Choose the number of neighbours k
2. Calculate the distance between the test point and all training data points.
   Euclidean Distance Formula:
   $d = \sqrt{(x_1-y_1)^2+(x_2-y_2)^2+\dots+(x_n-y_n)^2}$
3. Sort all distances in ascending order
4. Select the k-nearest neighbours
5. Decision:
   Classification → choose majority class
   Regression → take average of values.

Code (Implementation – Wine Dataset)

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbours import KNeighboursClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

wine = load_wine()
X = wine.data
Y = wine.target
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size = 0.3,
                                                    random_state = 42)
knn = KNeighboursClassifier(n_neighbours = 5)
knn.fit(x_train, y_train)
y_pred = knn.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred)).
```

→ model = make_pipeline(StandardScaler(), KNeighboursClassifier(n_neighbours

Multilinear Regression Algorithm Steps:

1. Collect dataset with multiple features
2. Define:
   Independent variables → x
   Dependent variable → y
3. Split dataset into training & testing sets
4. Train regression model:
   Estimate coefficients ($b_0, b_1, \dots, b_n$)
   Uses Least Squares method to minimize error.
5. Predict output values
6. Evaluate performance using:
   R^2 score
   Mean Squared Error (MSE)

Implementation:

```
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score.
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns = housing.feature_names)
df['Price'] = housing.target
print("Sample data:\n")
print(df.head())
X = df.drop('Price', axis=1)
y = df['Price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2
                                                    random_state = 42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
results = pd.DataFrame({
    'Actual': y_test[:5],
    'Predicted': y_pred[:5]
})
print("\nSample Predictions:\n")
print(results)
print("\nR2 Score:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
```

op:

Sample data:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Lat
0	8.3252	41.0	6.984	1.023	322.0	2.55	3
1	8.3019	21.0	6.23	0.97	2401.0	2.10	3
2	7.2574	52.0	8.28	1.073	496.0	2.80	37
3	5.6431	52.0	5.81	1.073	558.0	2.54	37
4	3.8462	52.0	6.28	1.081	565.0	2.181	37

Longitude	Price
-120.23	4.52
-122.22	3.58
-122.24	3.521
-122.25	3.413
-122.25	3.422

Sample predictions:

	Actual	Predicted
20046	0.477	0.719
3024	0.45	1.78
15663	5.00001	2.7096
20484	2.186	2.838
9814	2.78	2.6045

R2 Score : 0.5757

MSE : 0.555

Code:

```
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

y_pred_lr = lin_reg.predict(X_test_scaled)

rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

rmse_lr, mae_lr, r2_lr
```

```
(np.float64(0.7449540928043491), 0.5330139776645934, 0.5765012224420747)
```

```
dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

rmse_dt
```

```
np.float64(0.7138377373947808)
```

```
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

rmse_rf
```

```
np.float64(0.5128644706054181)
```

```
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    rf,
    param_grid,
    cv=3,
    scoring='neg_mean_squared_error'
)
```

```python
results = pd.DataFrame({
    'Model': ['Linear Regression',
              'Decision Tree',
              'Random Forest',
              'Tuned Random Forest'],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
})

results
```

	Model	RMSE
0	Linear Regression	0.744954
1	Decision Tree	0.713838
2	Random Forest	0.512864
3	Tuned Random Forest	0.513263

Program 5

Build Logistic Regression Model for a given dataset

Screenshot

Lab-06

Logistic Regression Model:

Upload dataset

```
from google colab import files
uploaded = files.upload()
```

Import Libraries:

```
import pandas as pd
import numpy as np

df = pd.read_csv('zoo.data(1).csv')
df.head()
```

Data Preprocessing

```
print(df.columns)
df = df.drop('animal_name', axis=1)
x = df.drop('animal_name', axis=1)
x = df.drop('class_type', axis=1)
y = df['class_type']
```

Train Test Split

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
                                                    random_state=42)
```

Train Logistic Regression model

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=500)
model.fit(x_train, y_train)
```

Predictions:

```
y_pred = model.predict(x_test)
```

Evaluate Model

```
from sklearn.metrics import accuracy_score, classification_report
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

o/p

accuracy: 0.95238095

classification report:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	12
2	1.00	1.00	1.00	2
3	0.00	0.00	0.00	1
4	0.67	1.00	0.87	2
5				
6	1.00	1.00	1.00	3
7	1.00	1.00	1.00	1
accuracy			0.95	21
macro avg	0.78	0.83	0.80	21
weighted avg	0.92	0.95	0.93	21

```
result = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_pred
})
```

```
result.head(6)
```

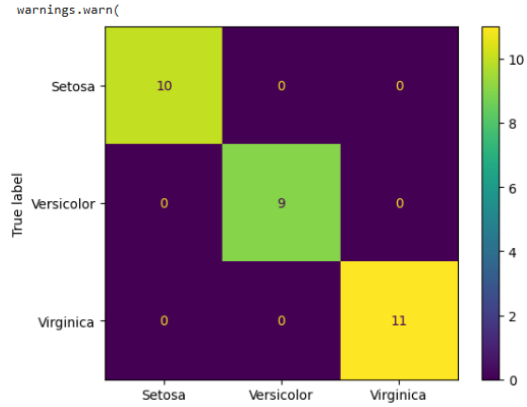
Actual	Predicted
1	1
1	1
1	1
1	1
1	1
6	6

Code:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn import metrics
import matplotlib.pyplot as plt

# Load correct file
iris = pd.read_csv("iris (1).csv")

print(iris.head())

# Features and target
X = iris.drop('species', axis='columns')
y = iris['species']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```python
# Confusion Matrix
cm = metrics.confusion_matrix(y_test, y_pred)
disp = metrics.ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Setosa", "Versicolor", "Virginica"]
)
disp.plot()
plt.show()
```

```
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
Accuracy: 1.0
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated i
  warnings.warn(
```



Program 6

Build KNN Classification model for a given dataset

Screenshot

Boosting Ensemble and
K-means

AdaBoost classifier:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
x = data.data
y = data.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_split=0.2
                                                    random_state=42)

model = AdaBoostClassifier(n_estimators=50, learning_rate=1.0,
                           random_state=42)

model.fit(x_train, y_train)

y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\n Classification Report:\n", classification_report(y_test,
                                                            y_pre
```

o/p:

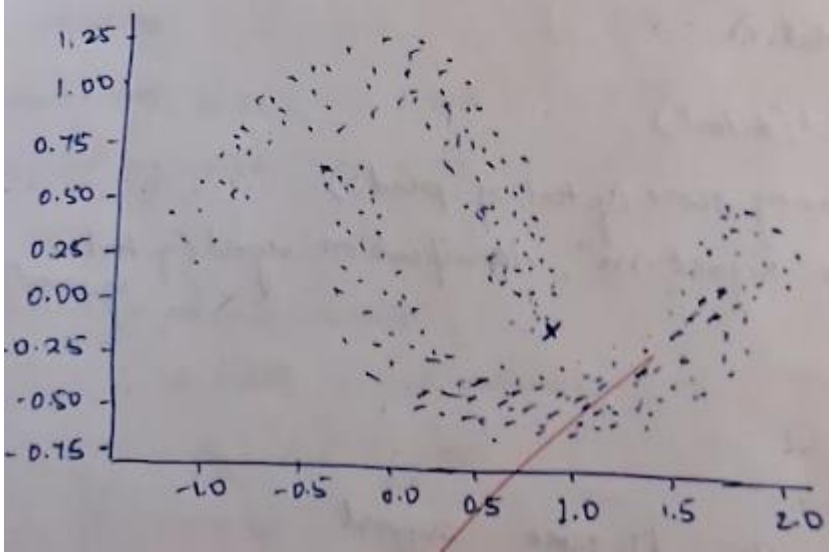
Accuracy: 0.9649122

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

2. K-Means Lab-04

```
import matplotlib.pyplot as plt
from sklearn.dataset import make_moons
from sklearn.cluster import KMeans

X,y = make_moon (n_samples =300, noise = 0.1, random_state =

kmeans = KMeans (n_clusters=2, random_state = 42)
kmeans.fit(X)

labels = kmeans.labels_

plt.scatter (X[:,0], X[:,1], c = labels)
plt.scatter (kmeans.cluster_centers_[:,0], kmeans.cluster_centers_
                                        [:,1], marker =
plt.title("K-means on Moons Dataset (Non-linear Data)")
plt.xlabel ("Feature 1")
plt.ylabel ("Feature 2")
plt.show ()
```



Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import mean_squared_error, r2_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

```
from sklearn.datasets import load_wine

data = load_wine()

X = data.data
y = data.target
```

```
from sklearn.datasets import load_diabetes

data = load_diabetes()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
regressor = DecisionTreeRegressor()

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)
```

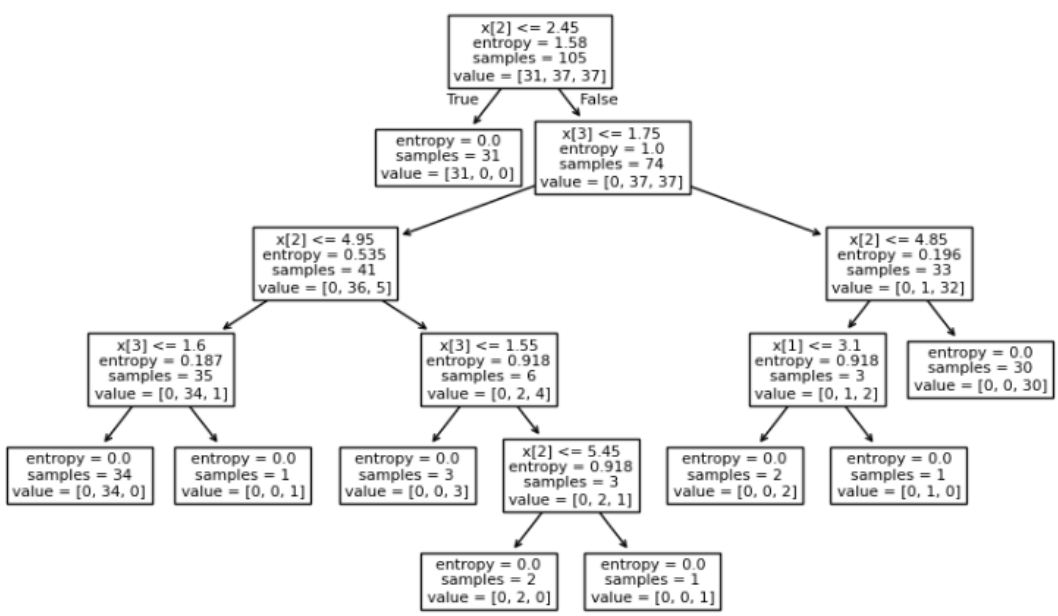
```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```
Mean Squared Error: 5358.458646616541
R2 Score: 0.0073817790735694055
```

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()

X = data.data
y = data.target
```

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_tree(model)
plt.show()
```

...



Program 7

Build Support vector machine model for a given dataset

Screenshot

Lab-08

SVM & Random Forest

1) SVM

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size = 0.2, random_state = 42
)
model = SVC (kernel = 'linear')
model.fit (X_train, y_train)
y_pred = model.predict (X_test)
print ("Accuracy: ", accuracy_score (y_test, y_pred))
print ("\n Classification Report:\n", classification_report(y_test, y_
```

op:

Accuracy: 0.95614035

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.91	0.94	43
1	0.95	0.99	0.97	71
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

Code:

```python
# Import libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Load dataset
data = datasets.load_iris()
X = data.data
y = data.target

# Feature scaling (important for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset (fixed for same results every time)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```python
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
cr = classification_report(y_test, y_pred)
print("\nClassification Report:\n", cr)
```

```
Accuracy: 0.9666666666666667

Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot

Random Forest: Lab-09

```
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.model_selection import test_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

data = load_digits()
X = data.data
y = data.target
x_train, x_test, y_train, y_test = train_test_split(
        X, y, test_size = 0.2, random_state = 42
)
model = RandomForestClassifier(n_estimators = 100, random_state = 42)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9722222

Classification Report:
              precision    recall  f1-score   support
           0       1.00      0.97      0.98        33
           1       0.97      1.00      0.98        28
           2       1.00      1.00      1.00        33
           3       1.00      0.94      0.97        34
           4       0.98      1.00      0.99        46
           5       0.94      0.96      0.95        47
           6       0.97      0.97      0.97        35
           7       0.97      0.97      0.97        34
           8       0.97      0.97      0.97        30
           9       0.95      0.95      0.95        40
    accuracy                           0.97       360
   macro avg       0.97      0.97      0.97       360
weighted avg       0.97      0.97      0.97       360
```

Code:

```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot

Boosting Ensemble and
K-means

AdaBoost classifier:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
x = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_split(x, y, test_split=0.2
                                                    random_state=42)

model = AdaBoostClassifier(n_estimators=50, learning_rate=1.0,
                           random_state=42)

model.fit(x_train, y_train)

y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\n Classification Report:\n", classification_report(y_test,
                                                            y_pre
```

O/P:

Accuracy: 0.9649122

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

Code:

```python
# Import libraries
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load Wine dataset
data = load_wine()
X = data.data
y = data.target

# Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
    n_estimators=50,      # number of weak learners
    learning_rate=1.0,    # contribution of each classifier
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```python
# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9444444444444444

Classification Report:
               precision    recall  f1-score   support

           0       0.93      1.00      0.97        14
           1       0.93      0.93      0.93        14
           2       1.00      0.88      0.93         8

    accuracy                           0.94        36
   macro avg       0.95      0.93      0.94        36
weighted avg       0.95      0.94      0.94        36
```

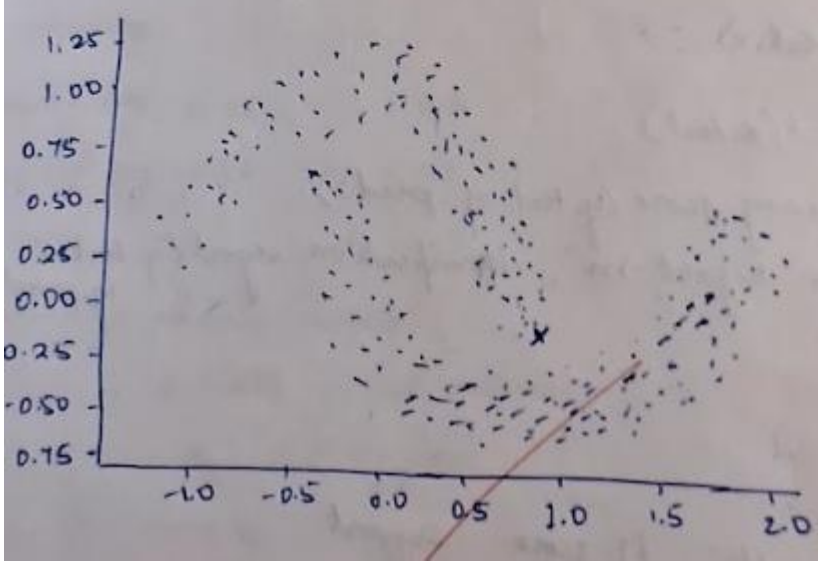
Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot

2. K-Means

```
import matplotlib.pyplot as plt
from sklearn.dataset import make_moons
from sklearn.cluster import KMeans

X,y = make_moon (n_samples =300, noise = 0.1, random_state =

Kmeans = KMeans (n_clusters=2. random_state = 42)
Kmeans.fit(X)

labels = kmeans.labels_

plt.scatter (X[:,0], X[:,1], c = labels)
plt.scatter (kmeans.cluster_centers_[:,0], kmeans.cluster_centers_
                                        [:,1], marker =
plt.title("K-means on Moons Dataset (Non-linear Data)")
plt.xlabel ("Feature 1")
plt.ylabel ("Feature 2")
plt.show ()
```



Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
data = pd.read_csv("/content/data.csv")
print(data.head())
```

```
X = data.iloc[:, :].values   # or choose specific columns
```

```
k = 3   # you can change this
```

```
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(X)
```

```
labels = kmeans.labels_
print(labels)
```

```
centroids = kmeans.cluster_centers_
print(centroids)
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='red')
plt.title("K-Means Clustering")
plt.show()
```

Program 11

Implement Dimensionality reduction using Principle Component Analysis (PCA) method.

Screenshot

PCA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.datasets.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data= load_breast_cancer (as_frame = True)
df = data.frame
df.head()
```

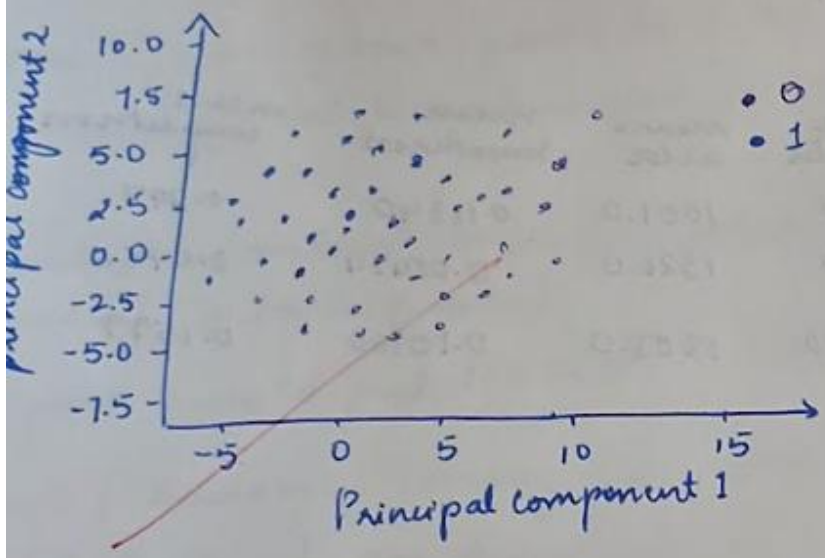
	mean radius	mean texture	mean parameter	mean area	mean smoothness	mean compactness
0	17.99	10.38	122.80	1001.0	0.11840	0.277
1	20.57	17.77	132.90	1326.0	0.08474	0.0786
2	19.69	21.25	130.00	1203.0	0.10960	0.1599

```
X = df.drop('target', axis = 1)
y = df['target']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components = 2)
X_pca = pca.fit_transform(X_scaled)
print ("Explained Variance Ratio: ", pca.explained_variance_rati
```

o/p: Explained Variance Ratio: [0.4427 0.18971]

```
pca_df = pd.DataFrame (data = X_pca, columns = ['PC1', 'PC2'])
pca_df['target'] = y
```

```
plt.figure()
for label in pca_df['target'].unique():
    subset = pca_df[pca_df['target'] == label]
    plt.scatter(subset['PC1'], subset['PC2'], label = label)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA visualization")
plt.legend()
plt.show()
```



Code:

PCA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

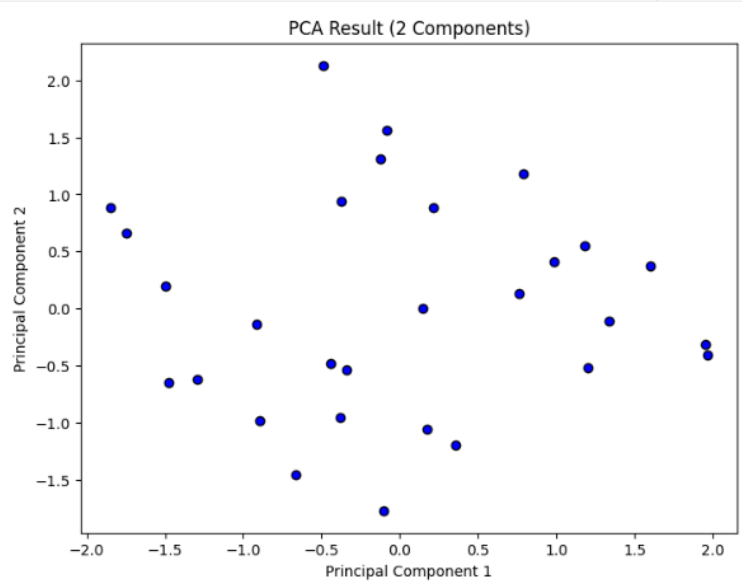
```
# Load dataset
data = pd.read_csv("data.csv")
print(data.head())
```

```
    x   y
0  30  49
1  31  27
2  31  46
3   1  65
4  81  76
```

```
# Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

```
# Apply PCA
pca = PCA(n_components=2)  # Reduce to 2 components
pca_result = pca.fit_transform(scaled_data)
```

```
# Explained variance ratio
print(f"Explained Variance Ratio: {pca.explained_variance_ratio_}")
print(f"Total Variance Explained by 2 Components: {np.sum(pca.explained_variance_ratio_)}")
```

```
Explained Variance Ratio: [0.56506606 0.43493394]
Total Variance Explained by 2 Components: 1.0
```

```
# Scatter plot of the 2D PCA result
plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], c='blue', edgecolors='black')
plt.title('PCA Result (2 Components)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```